

Отчёт по практической работе №2

Тема: качество распознавания и скорость обучения сети

Группа ЕТ-442, Замятин Андрей Александрович

1. Задание

По заданию (task.txt) нужно было: сначала письменно предположить, какие параметры обучения влияют на качество и влияет ли топология; затем на программе из практики 1 (network.py, класс Network, сигмоида, квадратичная ошибка, SGD; mnist_loader.py, mnist.pkl.gz: 50000 обучение / 10000 тест) обучить несколько сетей с разной топологией, для каждой подобрать параметры запуска; включить в отчёт не менее 8 тестов и ответить на три вопроса: (1) максимальная точность и топология; (2) влияние топологии; (3) влияние параметров.

2. Предположения до запусков (записано до запусков)

Скорость обучения: слишком большая ломает обучение — шаги огромные, веса прыгают, точность остаётся на уровне угадывания; слишком маленькая за 30 эпох не успевает обучить сеть. Оптимум ожидался около 3.0, как в практике 1, где это значение работало хорошо.

Размер мини-пакета: маленький (10) шумит, но обновлений за эпоху много ($50000/10 = 5000$ шагов), и сеть быстрее находит хорошее решение; большой (1000) считает градиент точнее, но обновлений за эпоху всего 50, поэтому обучение медленное. Число эпох: сначала точность растёт, потом выходит на плато, возможен небольшой откат.

Топология: скрытый слой нужен, без него сеть почти линейная и цифры разделит плохо. Мало нейронов в скрытом слое даёт недообучение — сети не хватит ёмкости запомнить варианты написания цифр. Много нейронов даёт больше ёмкости, но обучается дольше. Ожидалось, что 100 нейронов будут лучше 30, но большой сети может понадобиться меньшая скорость обучения.

3. Как запускал

- Рабочий каталог — practical-2 (загрузчик открывает mnist.pkl.gz по относительному имени). Редактор VS Code, запуск из терминала.
- Файл run_tests.py: список TESTS (название, топология, эпохи, пакет, скорость). Без аргументов — все тесты, с аргументами — только выбранные, например: `python3 -u run_tests.py T1`. Флаг `-u` — чтобы вывод сразу попадал в лог.
- Тесты T1-T9 выполнены по одному разу без фиксации seed и без замера времени. Это ограничение: разброс между запусками для них не измерялся, времени в их логах нет.
- Дополнительно выполнены T10-T15 и повторы тестов T1 (seed 1, 2) и T2 (seed 0) с фиксированным seed (`random + numpy`), замером времени (`time.perf_counter`) и распределением предсказаний по цифрам после обучения (`np.bincount`). Запуски выполнялись параллельно по несколько процессов.
- Проверка инициализации `check_init.py` считает модуль предактиваций $|z|$ выходного слоя и распределение предсказаний до обучения на первых 2000 примерах тестовой выборки (seed 0). Вывод скрипта сохранён в `logs/check_init.log` и в `results.txt`.
- Тест T15 — дополнительный эксперимент вне методички: сеть [784,100,10], пакет 10, $\eta=3.0$, seed 0, с масштабированной инициализацией весов ($w / \sqrt{\text{fan-in}}$) отдельной веткой в `run_one`. Файл `network.py` при этом не менялся. Лог — `logs/T15.log`, сводка — в `results.txt`.
- В `network.py` демо-сеть [2,3,1] из методички вынесена под `if __name__ == '__main__'`, поэтому в логах T1-T9 в начале есть её распечатка. Строка Defaults есть в логах T10-T14 и в логах повторов, а запуски после её удаления из `run_tests.py` (T15, `check_init`) её не содержат. На обучение это не влияет.
- Все логи — в папке `logs/`, сводка — в `results.txt`. Предупреждение `overflow` встречается только один раз — в логе T5 (скорость 100); в логе T4 его нет.

4. Сводная таблица (по results.txt и logs/)

Эпоха 0 / Макс. (номер эпохи) / Финал — число верных из 10000. Время — ориентировочное: замерено при одновременном запуске нескольких процессов, поэтому для сравнения топологий годится только как порядок величины. Для T1–T9 в исходных логах времени не было, там прочерк; ниже даны замеры из повторов на том же коде.

Тест	Топология	Пак.	eta	Эп.0	Макс.	Финал	Время
T1	[784,30,10]	10	3.0	9075	9527 (28)	9526	— *
T2	[784,100,10]	10	3.0	5658	8739 (28)	8729	— *
T3	[784,100,10]	1000	0.001	667	712 (29)	712	—
T4	[784,30,10]	10	100	974	1110 (1)	1110	—
T5	[784,10]	10	100	1951	3278 (28)	3191	—
T6	[784,30,10]	10	0.5	8400	9440 (28)	9424	—
T7	[784,30,10]	64	3.0	8117	9386 (28)	9385	—
T8	[784,100,10]	10	0.5	7690	8605 (27)	8602	—
T9	[784,30,10]	10	1.0	7989	9416 (27)	9406	—
T10	[784,10]	10	3.0	3696	8397 (27)	8382	93.0 с
T11	[784,10]	10	1.0	3514	7511 (29)	7511	92.4 с
T12	[784,100,10]	10	1.0	7324	8694 (29)	8694	555.4 с
T13	[784,50,10]	10	3.0	8726	9585 (24)	9579	300.9 с
T14	[784,15,10]	10	3.0	8864	9353 (29)	9353	140.7 с
T15	[784,100,10]	10	3.0	9441	9811 (29)	9811	505.9 с

* Замеры из повторов на том же коде с фиксированным seed: T1 seed1 — старт 8285, макс. и финал 9561 (29), 211.9 с; T1 seed2 — старт 9126, макс. 9539 (27), финал 9538, 213.1 с; исходный T1 — 9527/9526. Диапазон T1: 9527–9561 (разброс около 0.3%). T2 повтор seed0 — старт 8389 (против 5658 в исходном запуске), макс. 8744 (28), финал 8741, 540.4 с: старт сильно зависит от инициализации, итоговое плато — слабо (8739–8744).

Распределение предсказаний после обучения (из логов): T10 [784,10]: [1153, 1273, 1153, 1073, 1133, 1042, 1057, 1077, 5, 1034] — цифра 8 предсказана 5 раз; T11: [1197, 1254, 1605, 1092, 1202, 1440, 1, 1066, 5, 1138] — цифра 6 предсказана 1 раз, цифра 8 — 5 раз; T12 [784,100,10]: [1046, 1212, 1251, 1205, 1050, 1044, 1048, 1037, 0, 1107] — цифра 8 не предсказана ни разу; повтор T2: цифра 8 предсказана 2 раза. Для сравнения T13 [784,50,10]: [1009, 1146, 1036, 1029, 992, 857, 959, 1021, 972, 979], T14 [784,15,10]: [987, 1158, 1040, 993, 984, 914, 980, 1029, 946, 969] (диапазон примерно 900–1150) — без провалов по отдельным цифрам.

Проверка инициализации check_init.py (seed 0, 2000 примеров, без обучения; logs/check_init.log): [784,30,10] — среднее |z| 4.67, максимум 19.74, предсказания [0, 0, 1999, 0, 0, 0, 1, 0, 0, 0]; [784,100,10] — среднее |z| 5.62, максимум 23.50, предсказания [5, 0, 6, 2, 4, 0, 105, 194, 391, 1293]. До обучения предсказания схлопываются к одной-двум цифрам.

5. Что получилось по каждому тесту

Базовый тест и скорости на сети [784,30,10]: T1, T6, T9

T1 (eta=3.0): старт 9075, рост до максимума 9527 на 28-й эпохе, финал 9526. Повторы с seed 1 и 2 дали 9561 и 9539/9538 — диапазон 9527–9561. T6 (eta=0.5): старт 8400, максимум 9440 на 28-й эпохе, финал 9424. T9 (eta=1.0): старт 7989, максимум 9416 на 27-й эпохе, финал 9406. T6 (9440) и T9 (9416) различаются на 24 (0.24%), что в пределах разброса между запусками (у T1 разброс 34), поэтому скорости 0.5 и 1.0 считаем примерно равноценными; 3.0 лучше обеих (около 95.3–95.6% против около 94.2%).

Однослойная сеть: T5, T10, T11

T5 ([784,10], eta=100): старт 1951, колебания в диапазоне 2500–2900, максимум 3278 на 28-й

эпохе, финал 3191. Сравнение T1 и T5 само по себе ничего не говорит о нужности скрытого слоя: скорости разные (3.0 против 100), а при $\eta=100$ ломается даже сеть со скрытым слоем (T4: 1110). При обычных скоростях однослойная сеть учится: T10 ($\eta=3.0$) — старт 3696, максимум 8397 на 27-й эпохе, финал 8382; T11 ($\eta=1.0$) — старт 3514, максимум и финал 7511 на 29-й эпохе, кривая ещё росла. Училась однослойная сеть тяжело: в T10 долгое плато 5500–7600 до 15-й эпохи и прыжок на 16-й (7677–8349), в T11 прыжок на 26–27-й эпохах (6656–6881–7503). После обучения в T10 цифра 8 предсказана 5 раз, в T11 цифра 6 — 1 раз, цифра 8 — 5 раз. Прирост от скрытого слоя при $\eta=3.0$: около 9560 против около 8400 (примерно +1200).

Большая сеть [784,100,10]: T2, T8, T12

T2 ($\eta=3.0$): старт 5658, затем 7609 и 8439, максимум 8739 на 28-й эпохе, финал 8729. Повтор с seed 0 стартовал с 8389, дал максимум 8744 и финал 8741 — итоговое плато то же (8739–8744). T8 ($\eta=0.5$): старт 7690, максимум 8605 на 27-й эпохе, финал 8602; с 18-й эпохи значения стоят в диапазоне 8584–8605. T12 ($\eta=1.0$): старт 7324, максимум и финал 8694 на 29-й эпохе, 555 с. Порядок скоростей на этой топологии: 3.0 (около 8740) > 1.0 (8694) > 0.5 (8605) — снижение скорости не помогло. После обучения цифра 8 предсказана 0 раз (T12) и 2 раза (повтор T2). По данным диагностики, вероятно, насыщение сигмоиды; прямая проверка — тест T15, описанный ниже.

Прямая проверка: T15 ([784,100,10], масштабированная инициализация)

T15: старт 9441, на 1-й эпохе 9629, на 2-й — 9669, максимум 9811 на 29-й эпохе (финал тот же), время 505.9 с. Распределение предсказаний: [995, 1138, 1037, 1008, 983, 888, 959, 1036, 960, 996] — провала по цифре 8 нет (960 предсказаний). Вывод строго по результату: предположение о насыщении подтверждается — та же топология [784,100,10] при той же скорости 3.0, но с весами, поделёнными на $\sqrt{\text{fan-in}}$, дала 98.11% вместо плато около 87% и убрала провал по цифре 8.

Размер пакета: T1, T7 (и ограничение по T3)

T7 (пакет 64, $\eta=3.0$): старт 8117, максимум 9386 на 28-й эпохе, финал 9385 — ниже, чем T1 с пакетом 10 (9526–9561). Маленький пакет даёт точность выше. Тест T3 (пакет 1000, $\eta=0.001$, сеть [784,100,10]): 667–712, максимум 712 на 29-й эпохе. По T3 нельзя судить о влиянии одного только пакета: в нём одновременно изменены три параметра (скрытый слой 100, пакет 1000, скорость 0.001), а скорость 0.001 сама по себе почти не сдвигает веса (50 обновлений за эпоху). 667 из 10000 ниже уровня постоянного ответа (порядка 1000 при десяти классах), поэтому сеть не отвечает одним классом — она почти не учится.

Сломанные запуски: T4

T4 ([784,30,10], $\eta=100$): 974 на нулевой эпохе, 1110 с 1-й эпохи и далее все 30 эпох ровно 1110 — уровень угадывания. Шаг настолько большой, что сеть сразу ломается. Предупреждений overflow в его логе нет.

Новые топологии: T13, T14

T13 ([784,50,10], $\eta=3.0$): старт 8726, максимум 9585 на 24-й эпохе, финал 9579, 301 с — лучший результат среди тестов со стандартной инициализацией (см. T15 ниже). Распределение предсказаний ровное, без провалов. T14 ([784,15,10], $\eta=3.0$): старт 8864, максимум и финал 9353 на 29-й эпохе, 141 с. 15 нейронов хуже 30 и 50 при той же скорости.

6. Ответы на вопросы из задания

(1) Максимальная точность

9811 из 10000 (98.11%) — сеть [784,100,10], пакет 10, $\eta=3.0$ с масштабированной инициализацией (T15, дополнительный эксперимент), 29-я эпоха (финал тот же). Среди тестов со стандартной инициализацией максимум — 9585 (T13, [784,50,10], 24-я эпоха, финал 9579); среди первых девяти тестов — 9527 (T1, [784,30,10], 28-я эпоха). Повторы T1 с seed 1/2 дали 9561/9539.

(2) Влияние топологии

Скрытый слой помогает: при $\eta=3.0$ однослойная [784,10] — 8397, [784,15,10] — 9353, [784,30,10] — около 9530–9560, [784,50,10] — 9585. Но больше не значит лучше: [784,100,10] при всех проверенных скоростях (3.0/1.0/0.5) — 8605–8744 с провалом по цифре 8. По данным диагностики, вероятно, насыщение сигмоиды; прямая проверка (тест T15) описана ниже.

(3) Влияние параметров

Скорость 100 ломает обучение: T4 — 1110, T5 — 3278. На [784,30,10] скорость 3.0 (около 9530–9560) лучше, чем 0.5 (9440) и 1.0 (9416), а последние две между собой неразличимы в пределах разброса. На [784,100,10]: 3.0 (около 8740) > 1.0 (8694) > 0.5 (8605). Пакет: по паре T1–T7 пакет 10 (9526–9561) лучше пакета 64 (9385); пакет 1000 в связке со скоростью 0.001 (T3: 712) почти не учится. Разброс одиночных запусков без seed — порядка 0.1–0.3% (до 30 из 10000 незначимы), старт больших сетей гуляет сильнее.

7. Вывод и ограничения

Выполнено 15 конфигураций (+3 повтора и проверка инициализации), все логи сохранены. Лучший результат — 9811 (T15, [784,100,10] с масштабированной инициализацией); среди тестов со стандартной инициализацией — 9585 (T13, [784,50,10]). Предположения из раздела 2 подтвердились частично: про скорость (большая ломает, маленькая не успевает, оптимум около 3.0) и про пакет (маленький лучше, большой медленнее) — да; про 100 нейронов сразу лучше и про то, что меньшая скорость поможет большой сети, предположения не подтвердились: 100 нейронов дали 8605–8744 при всех скоростях.

Ограничения: по одному запуску на большинство тестов (кроме T1/T2), seed фиксирован только в дополнительных тестах; только сигмоида и квадратичная ошибка; время ориентировочное (замеры при параллельных запусках). Результат однослойной сети (около 84%) и сети со 100 нейронами могут зависеть от инициализации randn без масштабирования, поэтому сравнение с этими сетями приблизительное.

Исходный код: https://github.com/Trenj/deep_learning/tree/main/practical-2 — network.py, mnist_loader.py, run_tests.py, check_init.py, results.txt, logs/, этот отчёт.